BCC_S32K144_DualChain

Example project for Battery Cell Controller SW Driver
for MC33771C and MC33772C

Date: *15 December 2020*

Revision: *2.2*

Overview

The purpose of this example project is to demonstrate how to use the Battery Cell Controller (BCC) SW Driver for the communication with two TPL chains connected in parallel. Measured voltages from devices from both chains are printed to the console.

Hardware Requirements

Following is required:

- S32K144EVB-Q100 (MCU board)
- FRDMDUAL33664EVB (Board with two isolated network transceivers)
- 2pcs RD33771CDSTEVB or 2pcs KIT33772CTPLEVB (BCC boards)*
- 2pcs BATT-14CEMULATOR (Battery pack emulator for MC33771C) or
  2pcs BATT-6EMULATOR (Battery pack emulator for MC33772C) or
  other mean to supply the BCC EVBs, e.g. BATT-14EXTENDER.
- USB Micro B cable

*It is also possible to combine MC33771C and MC33772C EVBs after a simple code change in the example. See chapter *Import the Example Project* in this document.

Setting up Hardware

1. Make sure that you **disconnect** SBC (System Basis Chip) and power your S32K144EVB-Q100 from OpenSDA USB instead. SBC initialization and setting is not in scope of this example.

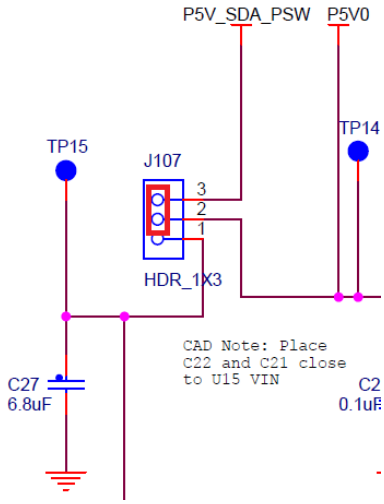

Figure 1. S32K144EVB-Q100 power source selection

2. Attach the FRDMDUAL33664EVB board to the MCU board.

3. Connect TPL IN (J3 connector) of one BCC board to the first TPL interface of FRDMDUAL33664EVB board (J1 connector) and TPL IN connector of the second BCC board to the other TPL interface of FRDMDUAL33664EVB board (J2 connector) by twisted pair cables.
4. Setup the battery pack emulators according to their user manual and connect them to the BCC boards using parallel cables.
5. Connect USB cable to USB interface on the MCU board and to your PC.

Figure 2 shows the complete HW setup for MC33771C EVBs and MC33771C battery emulators.

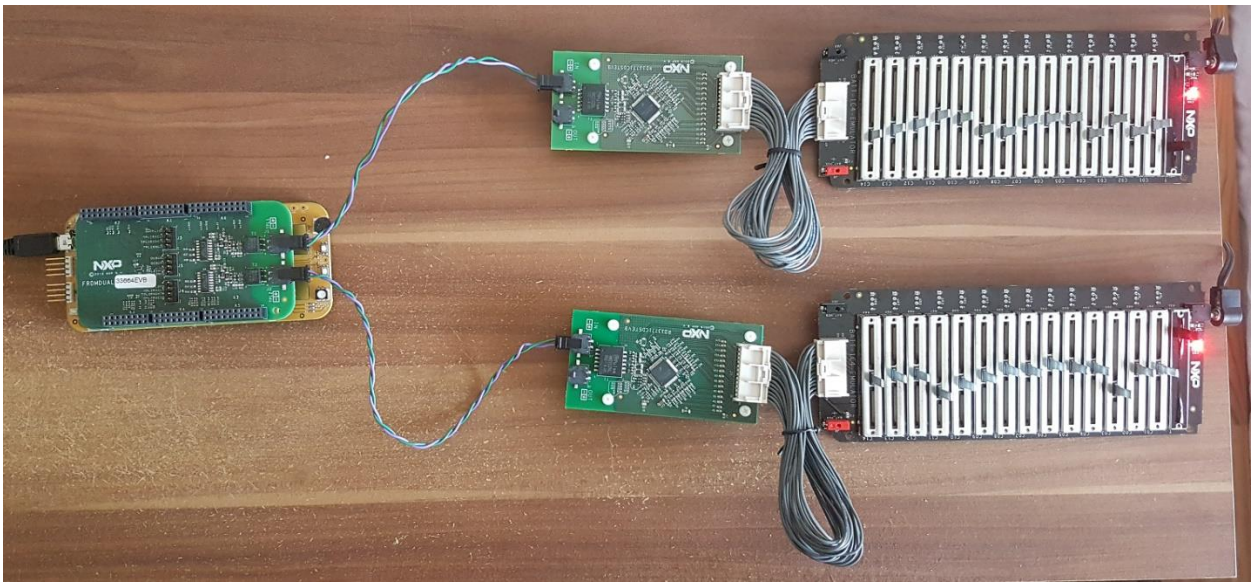

Figure 2. MC33771C HW setup

Setting up Software

Make sure that you have installed S32 Design Studio for ARM Version 2018.R1 with Update 11.

The application uses debug interface with virtual serial port to print user messages. Check that your debug connection has been set up properly. Type of used debug connection depends on used MCU. S32K144EVB-Q100 uses OpenSDA, see Figure 3. Note that number of COM port may differ because of different system resource usage. Baud rate is 115200 Bd.

Figure 3. OpenSDA virtual port

Description

The purpose of this example project is to demonstrate how to use the Battery Cell Controller (BCC) SW Driver for the communication with two TPL chains connected in parallel. Measured voltages from devices from both chains are printed to the console.

After the MCU peripherals initialization and BCC devices initialization, the program enters an infinite loop where the devices are waked up, on-demand conversion is performed, measured voltages from both BCC devices are printed to the console and the BCC devices are put into the sleep mode for a while.

When some error occurs during the initialization or in the further application run (e.g. CRC in the SPI frames is invalid), red LED on the MCU board starts to shine and the application run is stopped.

The project uses the following peripherals:

- LPSPI0 – SPI RX communication (TPL1 interface)
- LPSPI1 – SPI TX communication (both TPL1 and TPL2 interface)
- LPSPI2 – SPI RX communication (TPL2 interface)
- GPIO – INTB pins, EN pins, TPL1+2TXCSB pin and red onboard LED
- LPUART1 – serial communication
- LPIT0 – timeout for the BCC SW driver

Pin selection for all mentioned peripherals is listed in Table 1.

Table 1. Pin selection

Pin Function	S32K144 pin
TPL1TXDATA, TPL2TXDATA	PTB16/LPSPI1_SOUT
TPL1TXSCLK, TPL2TXSCLK	PTB14/LPSPI1_SCK
TPL1TXCSB	PTA6/LPSPI1_PCS1
TPL2TXCSB	PTA16/LPSPI1_PCS2
TPL1+2TXCSB	PTD3
TPL1RXDATA	PTB3/LPSPI0_SIN
TPL1RXSCLK	PTB2/LPSPI0_SCK
TPL1RXCSB	PTB5/LPSPI0_PCS1
TPL1EN	PTB0
TPL1INTB	PTB1
TPL2RXDATA	PTE16/LPSPI2_SIN
TPL2RXSCLK	PTE15/LPSPI2_SCK
TPL2RXCSB	PTA15/LPSPI2_PCS3
TPL2EN	PTD4
TPL2INTB	PTB12
Red on-board LED	PTD15
LPUART_RX	PTC6/LPUART1_RX

LPUART_TX	PTC7/LPUART1_TX

Example project consists of several files/folders apart from main.c as you can see in Figure 4.

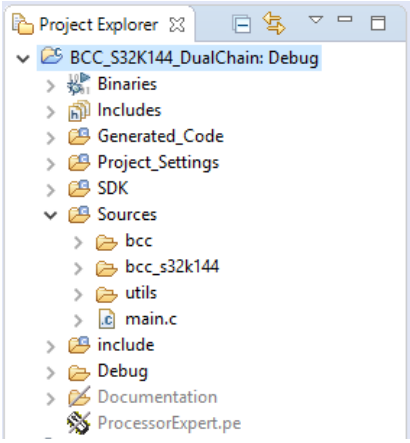

Figure 4. Example Code Structure

- **Sources/bcc** contains the BCC SW driver for MC33771C.
- **Sources/bcc_s32k144** contains external functions of BCC SW driver related to this example and S32K144 MCU.
- **Sources/utils** contains files used for serial communication.
- **Sources/main.c** contains the main functionality described above including the MCU peripherals initialization.

Additionally configuration of used MCU peripherals is done by Processor Expert components. Outputs of those are generated configuration structures (in *Generated_Code* folder) which are then passed to initialization functions in *main.c*.

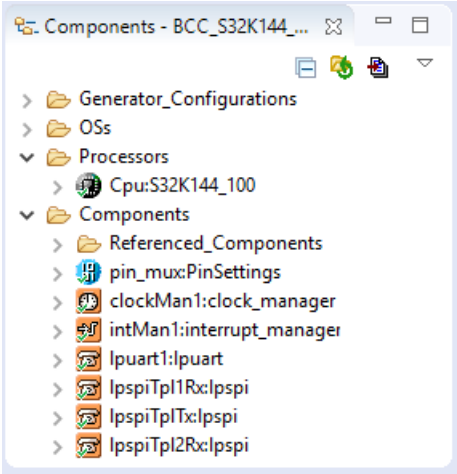

Figure 5. PEx components

Import the Example Project

The following steps shows how to import an example project into S32 Design Studio for ARM Version 2018.R1.

1. In S32 DS click on the ***File / Import***.
2. Choose ***General / Existing Projects into Workspace***.
3. Click ***Browse to select root directory*** with your downloaded example projects.
4. ***Select project*** named **BCC_S32K144_DualChain** and click ***Finish*** to complete the process.
5. At the top of **Sources/main.c** file define either macro **MC33771C** (in case of HW setup with two MC33771C) or **MC33772C** (in case of HW setup with two MC33772C).
6. Now the example project should be in your workspace and ready to run.

Note: It is also possible to mix the BCC devices (to use MC33771C in one chain and MC33772C in the other chain) when you change the BCC configuration (**bcc_drv_config_t** structure) in **initDemo()** in **main.c**.

Building and Running the Project

In order to build and run the project you need to ***build*** the project in a usual way. If the build is successful, ***debug and run*** the project. This can be accomplished in following steps:

1. Click on the **arrow** next to the **debug icon** and select **Debug Configurations**.
2. **Select** one of the existing configurations with **project name** under **PEMicro** group or **create** one by double clicking on this group.
3. Pick up proper **debug interface** and **USB port**.
4. Apply changes and click on **Debug**.

Note that the first build shows several warnings related to implicit declaration of math functions (Figure 6). These warnings have no impact on example functionality and will disappear after the next build.

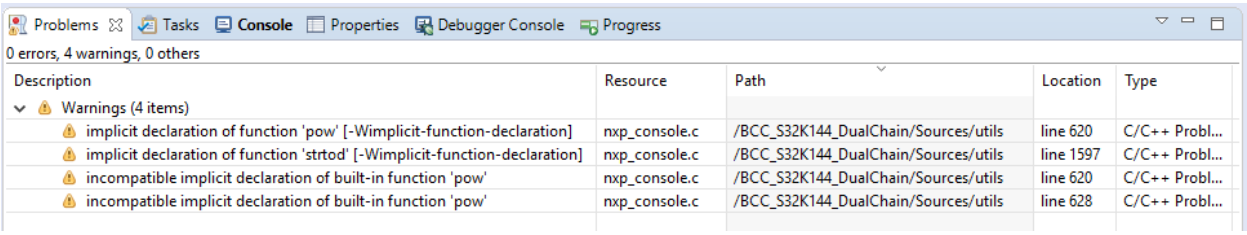

Figure 6. Warnings after the first build

If you have any questions related to how to work with debug configurations, see ***S32 Design Studio User's Guide***.